Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Регламент переключения на реплику

Авария основного сервера и возврат · версия 1.0.0

30 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Когда применять 4

3 Перед началом 5

4 Переключение 6

4.1 1. Зафиксируйте состояние 6

4.2 2. Остановите основной сервер 6

4.3 3. Повысьте реплику 6

4.4 4. Переключите приложение и копирование 7

4.5 5. Закрепите адрес базы в `.env` 7

4.6 6. Сообщите о восстановлении работы 7

5 Проверка 8

5.1 1. Приложение работает с новым основным сервером 8

5.2 2. Вход, график и запись 8

5.3 3. Архив WAL и копии нового сервера 8

6 Архив WAL после переключения 9

7 Работа без реплики 10

8 Возврат к схеме с репликой 11

8.1 1. Сделайте копию и запишите контрольные числа 11

8.2 2. Сохраните том бывшего основного сервера (если нужно для разбора) 11

8.3 3. Остановите приложение и новый основной сервер 11

8.4 4. Перенесите данные в том основного сервера 12

8.5 5. Запустите основной сервер на перенесённых данных 12

8.6 6. Верните адрес базы 12

8.7 7. Создайте реплику заново 12

8.8 8. Запустите стенд 12

8.9 9. Проверьте схему с репликой 13

9 Если не получилось 14

10 Запись в журнал эксплуатации 15

1 О документе

Регламент переключения «Планировщика отпусков» на реплику PostgreSQL при аварии основного сервера (E52-04): переключение, проверка, работа без реплики, обращение с архивом WAL и возврат к схеме с репликой. Применяется на стенде, запущенном с `docker-compose.replica.yml`. Справочник — «Руководство по развёртыванию», разделы «Реплика и архив WAL» и «Переключение на реплику»; решение — ADR-014 (RPO — секунды, RTO — около 15 минут).

Шаги переключения повторяют задание CI «Реплика PostgreSQL, архив WAL и переключение» (`.github/workflows/ci.yml`), которое на каждый push останавливает основной сервер, повышает реплику, переводит на неё `server` и проверяет вход, чтение, запись и архив WAL новой линии времени. Возврат к схеме с репликой в CI не проверяется — отработайте его на тестовом стенде до первого применения.

Команды выполняются на узле стенда из каталога, где лежат файлы compose и `.env`. Время в журналах — UTC.

2 Когда применять

Переключайтесь, только если выполнены все условия:

1. Основной сервер `postgres` не работает: контейнер не запускается, том `db-data` повреждён или недоступен диск с ним (оповещение `VacationDatabaseDown`, `$C ps postgres`, `$C logs postgres`).
2. Вернуть его в строй за 15 минут нельзя: перезапуск не помогает, причина — в диске, томе или данных.
3. Реплика работает: `postgres-replica` в состоянии `running`, до аварии была `replica1|streaming` и не было оповещения `VacationReplicaLagging`.

СИТУАЦИЯ	ЧТО ДЕЛАТЬ ВМЕСТО ПЕРЕКЛЮЧЕНИЯ
`postgres` перезапустился и отвечает	Ничего; разбор причины — `alerts.md`, `VacationDatabaseDown`
Ошибочное массовое изменение данных	Реплика уже повторила его — `backup-restore.md`, «Восстановление на момент времени»
Недоступен весь узел стенда	Реплика из `docker-compose.replica.yml` работает на том же узле — восстановление на новом узле по `backup-restore.md`
Реплика тоже не работает или отстала на часы	Восстановление из копий — `backup-restore.md`

Решение о переключении принимает ответственный за систему: после повышения реплики бывший основной сервер в строй не возвращается.

3 Перед началом

ЧТО НУЖНО	ПОДРОБНОСТИ
Доступ	SSH к узлу стенда, право запускать `docker`, правка `.env`
Кто	Администратор стенда выполняет; ответственный за систему принимает решение; кадровая служба получает сообщение о простое и возможной потере последних секунд изменений
Время	Переключение — 15 минут; возврат к схеме с репликой — плановое окно 30–60 минут
Переменные	В `.env` заданы `JWT_SECRET` и `REPLICATION_PASSWORD` — без них compose не выполнит ни одной команды с этими файлами

```
C="docker compose -f docker-compose.yml -f docker-compose.replica.yml"
```

4 Переключение

4.1 1. Зафиксируйте состояние

```
date -u
$C ps postgres postgres-replica
$C logs --timestamps --since 1h postgres > postgres-before-failover-$(date +%Y%m%d-%H%M).log
$C exec postgres-replica psql -U vacation -Atc "select pg_is_in_recovery(),
pg_last_wal_replay_lsn(), pg_last_xact_replay_timestamp()"
$C exec postgres-replica psql -U vacation -d vacation -Atc "select count(*) from employees"
```

Ожидаемый результат: реплика отвечает `t|<позиция WAL>|<время последней применённой транзакции>`. Изменения, зафиксированные на основном сервере позже этого времени, если они были, потеряны — запишите время в журнал и сообщите кадровой службе. Журнал `postgres` сохранён для разбора причины.

4.2 2. Остановите основной сервер

```
$C stop postgres          # если контейнер ещё работает или перезапускается
$C kill postgres          # если stop не завершился
$C ps postgres
```

Ожидаемый результат: `postgres` — `exited`. Пока реплика работает основной, бывший основной сервер не запускается: два основных сервера разойдутся, а старый допишет в общий архив WAL сегменты своей линии времени.

Сервисы стенда запускаются с `restart: unless-stopped` (с версии 1.0.0): упавший контейнер Docker поднимает сам, в том числе после перезагрузки узла. Остановленный командой `stop` или `kill` контейнер остаётся остановленным и после перезагрузки — поэтому основной сервер останавливают именно так, а не ждут, пока он упадёт. Любой `up` без `--no-deps` запустит его снова (от него зависят `server` и `backup`) — до возврата к схеме с репликой используйте только команды этого регламента.

4.3 3. Повысьте реплику

```
$C exec postgres-replica psql -U vacation -c "select pg_promote()"
$C exec postgres-replica psql -U vacation -Atc "select pg_is_in_recovery()"
```

Ожидаемый результат: `pg_promote` — `t`; `pg_is_in_recovery()` — `f` (в течение нескольких секунд, повторите запрос, если ещё `t`). Реплика стала основным сервером и принимает запись.

4.4 4. Переключите приложение и копирование

```
DB_HOST=postgres-replica $C --profile full up -d --no-deps server backup basebackup
```

Ожидаемый результат: `server`, `backup` и `basebackup` пересозданы с `DB_HOST=postgres-replica`. `--no-deps` обязателен: в файлах compose `server`, `backup` и `postgres-replica` зависят от `postgres`, и без флага compose запустил бы старый основной сервер. `web` не перезапускается — nginx заново находит адрес `server` каждые 10 секунд.

4.5 5. Закрепите адрес базы в `.env`

Добавьте в `.env` строку `DB_HOST=postgres-replica` и проверьте:

```
grep '^DB_HOST=' .env       # DB_HOST=postgres-replica
```

Без неё следующий `up` вернёт `server`, `backup` и `basebackup` на старый узел.

4.6 6. Сообщите о восстановлении работы

Кадровой службе — время простоя и время последней сохранённой транзакции из шага 1; дежурным по мониторингу — что стенд работает без реплики (раздел «Работа без реплики»).

5 Проверка

5.1 1. Приложение работает с новым основным сервером

```
$C exec server wget -qO- http://localhost:8080/readyz                        #
{"ok":true,"db":"up"}
curl -s http://localhost/api/v1/employees -o /dev/null -w '%{http_code}\n'   # 401 — API за
nginx отвечает
$C exec -T server sh -c 'wget -qO- --header "Authorization: Bearer $METRICS_TOKEN"
http://127.0.0.1:8080/metrics' \
  | grep -E '^vp_db_(up|wal_archive_enabled|wal_archive_failing) '
$C exec postgres-replica psql -U vacation -d vacation -Atc "select count(*) from employees"
```

Ожидаемый результат: `vp_db_up 1`, `vp_db_wal_archive_enabled 1`, `vp_db_wal_archive_failing 0`; число сотрудников — как в шаге 1 «Переключения».

5.2 2. Вход, график и запись

Сотрудник кадровой службы входит в интерфейс, открывает «График отпусков» и выполняет обычное действие с сохранением (например, решение по заявке) — изменение сохраняется и появляется в «Журнале аудита».

5.3 3. Архив WAL и копии нового сервера

```
$C exec postgres-replica psql -U vacation -Atc "select pg_switch_wal()"
$C exec postgres-replica sh -c 'ls /wal-archive | grep "\.history$"'
$C exec postgres-replica psql -U vacation -Atc "select last_archived_wal, last_failed_wal
from pg_stat_archiver"
$C logs --since 15m backup | grep 'backup:'
$C logs --since 15m basebackup | grep 'basebackup:'
```

Ожидаемый результат: в архиве файл новой линии времени (`00000002.history` после первого переключения), последний заархивированный сегмент начинается с её номера (`00000002…`), `last_failed_wal` пуст; `backup` и `basebackup` сразу после пересоздания сняли копии с нового основного сервера (`backup: /backups/vacation-….dump`, `basebackup: <метка>`). Физическая копия большой базы занимает минуты — повторите последнюю команду позже.

6 Архив WAL после переключения

ЧТО ПРОИСХОДИТ	ЧТО ЭТО ЗНАЧИТ ДЛЯ ЭКСПЛУАТАЦИИ
Повышенная реплика архивирует WAL в тот же том `wal-archive`, на новой линии времени: сегменты `00000002…`, файл развилки `00000002.history`; последний неполный сегмент старой линии — с суффиксом `.partial`	Архив продолжается без вмешательства; `basebackup.sh` никогда не удаляет `*.history`
Физические копии, снятые до переключения, остаются годными	Восстановление на момент времени проходит через развилку по файлу `.history` — до и после переключения (`backup-restore.md`)
`basebackup` пересоздан на шаге 4 и сразу снимает копию нового сервера	Через `BASEBACKUP_KEEP` суток в архиве останутся только копии новой линии
Бывший основной сервер, запущенный со своим томом, продолжил бы архивировать сегменты старой линии после развилки в тот же том	Не запускайте `postgres` до возврата (раздел ниже): его том перезаписывается данными нового основного
Авария уничтожила и том `wal-archive`	Восстановление на момент до переключения невозможно; окно восстановления начинается с первой физической копии нового сервера — проверьте `$C exec basebackup ls /wal-archive/base`. Немедленно сделайте внеочередную логическую копию и вынесите её с сервера

7 Работа без реплики

До возврата к схеме с репликой:

1. Потеря данных при аварии нового основного сервера — до 5 минут (архив WAL), а не секунды.
2. Оповещения `VacationReplicaDisconnected` и `VacationReplicaLagging` молчат: у нового основного сервера нет ни слота, ни реплики. Это не признак исправности. `VacationWalArchiveFailing` и `VacationDatabaseDown` работают.
3. Запускайте сервисы только с `--no-deps` и явным перечнем: `$C --profile full up -d --no-deps server web backup basebackup`. Команда `up` без `--no-deps` запустит старый `postgres`.
4. В командах других регламентов вместо `$C exec postgres …` выполняйте `$C exec postgres-replica …`.
5. Обновление версии (`update.md`) — только после возврата.
6. Запланируйте возврат на ближайшее окно работ.

8 Возврат к схеме с репликой

Бывший основной сервер не подключается обратно: его данные отстали и разошлись. Возврат переносит данные нового основного сервера (том `db-replica`) в том сервиса `postgres` (`db-data`) и создаёт реплику заново скриптом `deploy/postgres/replica.sh` — файлы compose не меняются, роли сервисов становятся прежними. Простой — на время копирования тома базы.

Ниже имена томов — с префиксом `vacation-planner_` (имя проекта compose, по умолчанию — имя каталога стенда); свои найдите командой `docker volume ls | grep -E 'db-data|db-replica'`.

8.1 1. Сделайте копию и запишите контрольные числа

```
$C exec backup sh -c 'pg_dump -Fc -f /backups/vacation-manual-$(date +%Y%m%d-%H%M).dump'
$C cp backup:/backups/<файл>.dump ./
$C exec postgres-replica psql -U vacation -d vacation -Atc "select count(*) from employees"
df -h "$(docker info -f '{{.DockerRootDir}}')"
docker system df -v | grep -E 'db-data|db-replica'
```

Ожидаемый результат: копия снята с нового основного сервера и вынесена с узла; свободного места больше размера тома `db-replica`.

8.2 2. Сохраните том бывшего основного сервера (если нужно для разбора)

По решению ответственного за систему или службы ИБ — до того, как том будет перезаписан:

```
docker run --rm -v vacation-planner_db-data:/data:ro -v /srv:/out postgres:17-alpine \
  tar czf /out/old-primary-db-data.tar.gz -C /data .
```

8.3 3. Остановите приложение и новый основной сервер

```
$C --profile full stop web server backup basebackup
$C stop -t 120 postgres-replica
$C logs --tail 3 postgres-replica
```

Ожидаемый результат: сервисы — `exited`; последняя строка журнала `postgres-replica` — «database system is shut down» (штатный останов, данные согласованы). С этого момента идёт простой.

8.4 4. Перенесите данные в том основного сервера

```
docker run --rm -v vacation-planner_db-replica:/from:ro -v vacation-planner_db-data:/to
postgres:17-alpine \
  sh -c 'rm -rf /to/* && cp -a /from/. /to/ && ls /to'
```

Ожидаемый результат: в списке `PG_VERSION`, `base`, `global`, `pg_wal` и нет `standby.signal` (его удалило повышение).

8.5 5. Запустите основной сервер на перенесённых данных

```
$C up -d --no-deps postgres
$C exec postgres psql -U vacation -Atc "select pg_is_in_recovery()"
$C exec postgres psql -U vacation -d vacation -Atc "select count(*) from employees"
$C exec postgres psql -U vacation -Atc "select pg_switch_wal()"
$C exec postgres psql -U vacation -Atc "select last_archived_wal, last_failed_wal from
pg_stat_archiver"
```

Ожидаемый результат: `f`; число сотрудников — как в шаге 1; архивируются сегменты текущей линии времени (`00000002…`), `last_failed_wal` пуст. Том `db-replica` пока не тронут — если что-то не так, переходите к разделу «Если не получилось».

8.6 6. Верните адрес базы

Удалите строку `DB_HOST=postgres-replica` из `.env`:

```
grep '^DB_HOST=' .env       # пусто
```

8.7 7. Создайте реплику заново

```
$C rm -f postgres-replica
docker volume rm vacation-planner_db-replica
```

Ожидаемый результат: контейнер и том удалены. Пока том не удалён, нельзя выполнять `up` без `--no-deps`: `postgres-replica` запустился бы на нём вторым основным сервером.

8.8 8. Запустите стенд

```
$C --profile full up -d
$C logs -f postgres-replica       # «replica: копия данных с postgres», затем готовность к
подключениям только на чтение
```

Ожидаемый результат: `server`, `web` и `backup` работают с `postgres` (простой закончился); `postgres-replica` создаёт роль и слот `replica1` на основном сервере, копирует данные и переходит в режим горячего резерва; после этого запускается `basebackup`. Если копирование дольше 5 минут, compose сообщит, что `basebackup` не дождался готовности реплики, — дождитесь конца копирования и выполните `$C up -d basebackup`.

8.9 9. Проверьте схему с репликой

```
$C exec postgres psql -U vacation -Atc "select application_name, state from
pg_stat_replication"   # replica1|streaming
$C exec postgres psql -U vacation -Atc "select archived_count, failed_count from
pg_stat_archiver"  # failed_count не растёт
$C exec basebackup ls /wal-archive/base
# новая копия по дате
$C exec server wget -qO- http://localhost:8080/readyz
# {"ok":true,"db":"up"}
```

Затем — вход, «График отпусков» и сохранение изменения в интерфейсе, как в разделе «Проверка». Оповещения `VacationReplicaDisconnected` и `VacationReplicaLagging` снова контролируют реплику.

9 Если не получилось

ПРИЗНАК	ДЕЙСТВИЕ
`pg_promote()` вернул ошибку «recovery is not in progress»	Реплика уже работает основным (повышена раньше) — проверьте `pg_is_in_recovery()` и продолжайте с шага 4
Реплика не повышается или не запускается	Журнал `$C logs postgres-replica`. Без исправной реплики — восстановление из копий (`backup-restore.md`): логическая копия или восстановление на момент времени в новый том; эскалация администратору БД
`server` после шага 4 не готов (`/readyz` не отвечает)	`$C exec server printenv DATABASE_URL` — в адресе должен быть `postgres-replica` ; журнал `$C logs --no-log-prefix server`
Возврат: `postgres` не стартует на перенесённых данных (шаг 5)	Том `db-replica` цел: `$C stop postgres` , верните `DB_HOST=postgres-replica` в `.env` , `$C up -d --no-deps postgres-replica` и `$C --profile full up -d --no-deps server web backup basebackup` — работа продолжается без реплики; журнал `postgres` — разработчикам и администратору БД
Возврат: новая реплика не создаётся, «password authentication failed for user “replicator”»	`REPLICATION_PASSWORD` в `.env` отличается от того, с которым создана роль, — верните прежний (раздел «Устранение неполадок» руководства)
Возврат: в журнале `postgres` «archive command failed»	`alerts.md` , `VacationWalArchiveFailing`

10 Запись в журнал эксплуатации

ПОЛЕ	ПРИМЕР
Время аварии, решения и восстановления работы (UTC)	Авария 11:02, решение 11:08, работа восстановлена 11:15
Кто принял решение, кто выполнял	Ответственный за систему; администратор стенда
Причина недоступности основного сервера	Диск тома `db-data`, повреждение данных, другое; ссылка на журнал `postgres-before-failover-…log`
Потеря данных	Время последней применённой транзакции на реплике из шага 1
Проверка	`readyz`, метрики, вход, запись, `00000002.history`, первая физическая копия нового сервера
Возврат к схеме с репликой	Дата окна, простой, контрольные числа, `replica1|streaming`